COMPILERS
(LECTURE 3, 4)

By :

Dr. Sherin Omran

Regular Expressions

- A set is called regular if it can be generated from the elements of the alphabet using union , concatenation and Kleene star operation.
- regular expressions has come into common use for describing all the languages that can be built from regular sets.

Example 3.4: Let $\Sigma = \{a, b\}$.

1. The regular expression **a|b** denotes the language $\{a, b\}$.

2. **(a|b)(a|b)** denotes $\{aa, ab, ba, bb\}$, the language of all strings of length two over the alphabet Σ. Another regular expression for the same language is **aa|ab|ba|bb**.

3. $\mathbf{a}^*$ denotes the language consisting of all strings of zero or more a's, that is, $\{\epsilon, a, aa, aaa, \ldots\}$.

4. $(\mathbf{a}|\mathbf{b})^*$ denotes the set of all strings consisting of zero or more instances of a or b, that is, all strings of a's and b's: $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \ldots\}$. Another regular expression for the same language is $(\mathbf{a}^*\mathbf{b}^*)^*$.

5. $\mathbf{a}|\mathbf{a}^*\mathbf{b}$ denotes the language $\{a, b, ab, aab, aaab, \ldots\}$, that is, the string a and all strings consisting of zero or more a's and ending in b.

Regular expression examples

Write the regular expression for the language starting and ending with a and having any having any combination of b's in between. Such that the alphabet is {a,b}

Solution:
The regular expression will be
R = a b* a

Regular expression examples

- Write the regular expression for the language starting with a but not having consecutive b's over the alphabet {a,b}.
- **Solution:** The regular expression has to be built for the language:

  L = {a, aba, aab, aba, aaa, abab, ...}

- The regular expression for the above language is: R = {a | ab}*

Regular expression examples

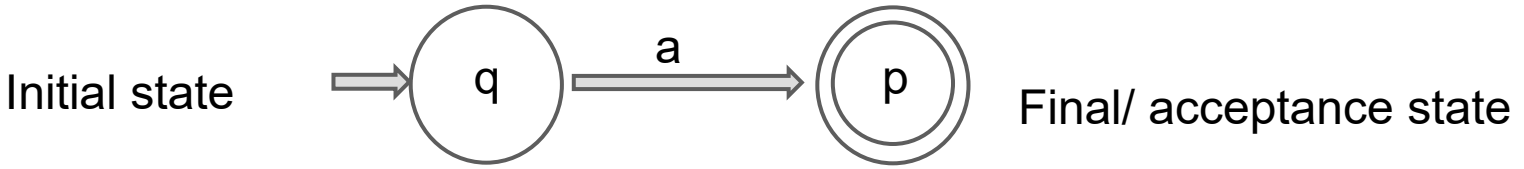
- Write a regular expression for the language that starts and ends with x over the alphabet { x, y, z}.

- L= { xx, xyx, xzx, xyzx, xzyx, xxyzx,……..}.

- The regular expression for the above language is R= x (x|y|z)* x.

Transition diagram

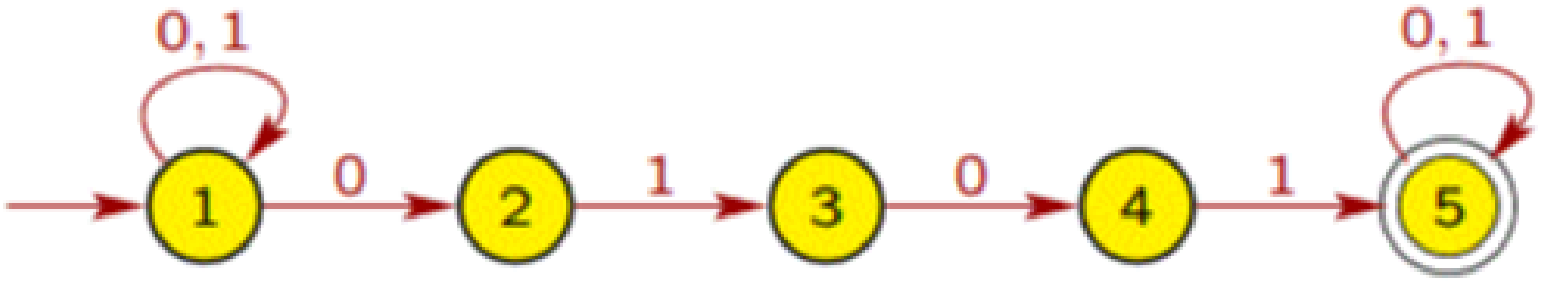
- A transition diagram or state transition diagram is a directed graph which can be constructed as follows:
- There is a node for each state, which is represented by the circle.
- There is a directed edge from node q to node p labeled a if the current state q received the input symbol a its next state will be state p.
- In the start state, there is an arrow with no source.
- One or more accepting states or final states are indicating by a double circle.



Transition diagram Cont.

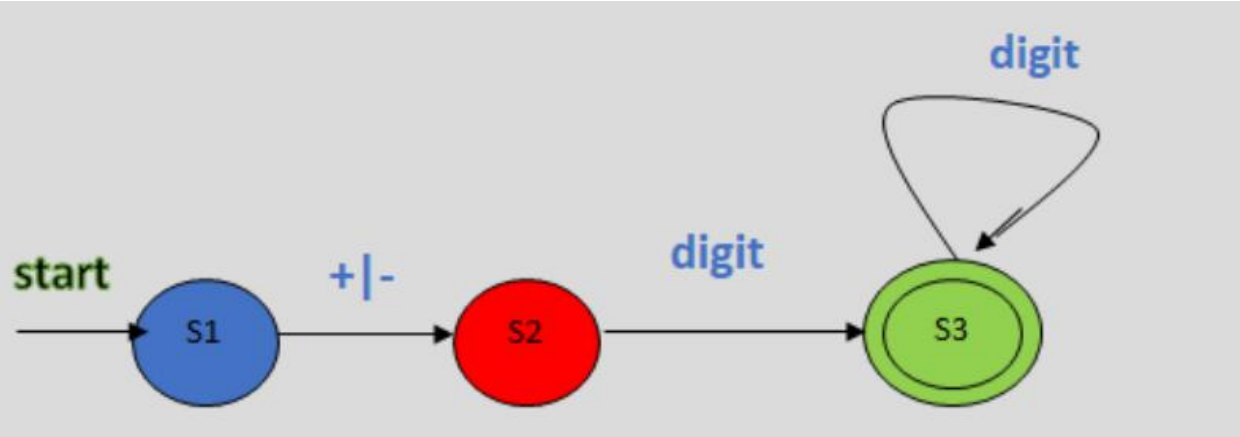
- Convert the following regular expression into a transition diagram:

(0|1)* 0101(0|1)*

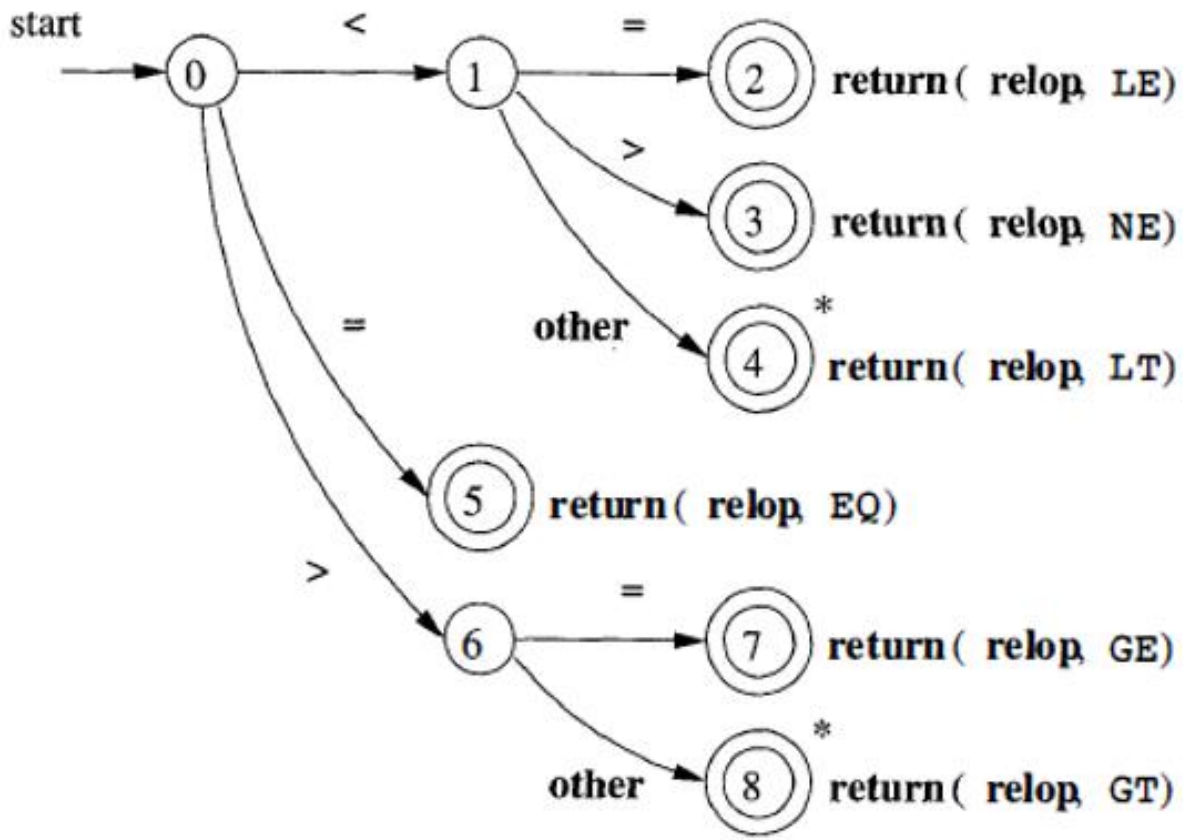


Transition diagram Cont.

- Question :
- Find the regular expression that represent the signed numbers such as +123. Then convert into a transition diagram
- Regular Expression (+|-) digit digit*



- Try to solve for the signed float / real numbers such as -39.37

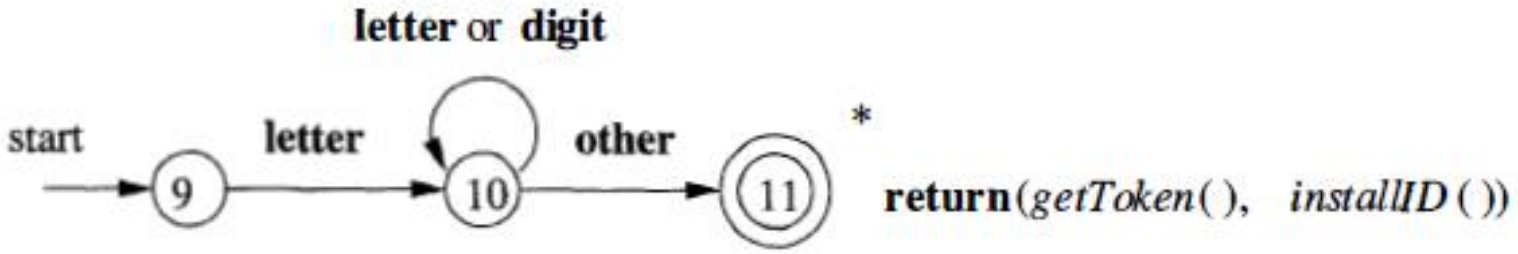


TRANSITION DIAGRAM FOR RELATIONAL OPERATORS

- The previous figure is a transition diagram that recognizes the lexemes matching the token relop.
- We begin in state 0 , the start state. If we see < as the first input symbol, then among the lexemes that match the pattern for relop we can only be looking at < , < > , or <=.
- We therefore go to state 1, and look at the next character.
- If it is =, then we recognize lexeme <=, enter state 2, and return the token relop with attribute LE, the symbolic constant representing this particular comparison operator.
- If in state 1 the next character is > , then instead we have lexeme <>, and enter state 3 to return an indication that the not-equals operator has been found.
- On any other character, the lexeme is < , and we enter state 4 to return that information. Note, however, that state 4 has a * to indicate that we must retract the input one position.

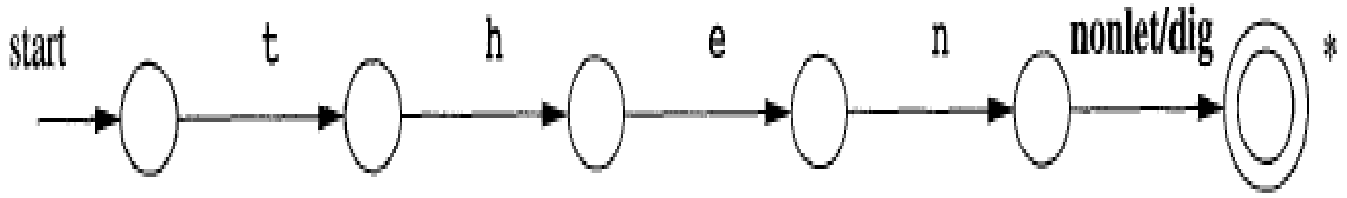
- On the other hand, if in state 0 the first character we see is =, then this one character must be the lexeme. We immediately return that fact from state 5.
- The remaining possibility is that the first character is > . Then, we must enter state 6 and decide, on the basis of the next character, whether the lexeme is >= (if we next see the = sign), or just > (on any other character).
- Note that if, in state 0, we see any character besides <, =, or >, we can not possibly be seeing a relop lexeme, so this transition diagram will not be used.

Transition diagram Cont.



- A transition diagram for identifiers and keywords

Transition diagram Cont.



- Hypothetical transition diagram for the keyword then

Transition diagram for unsigned numbers

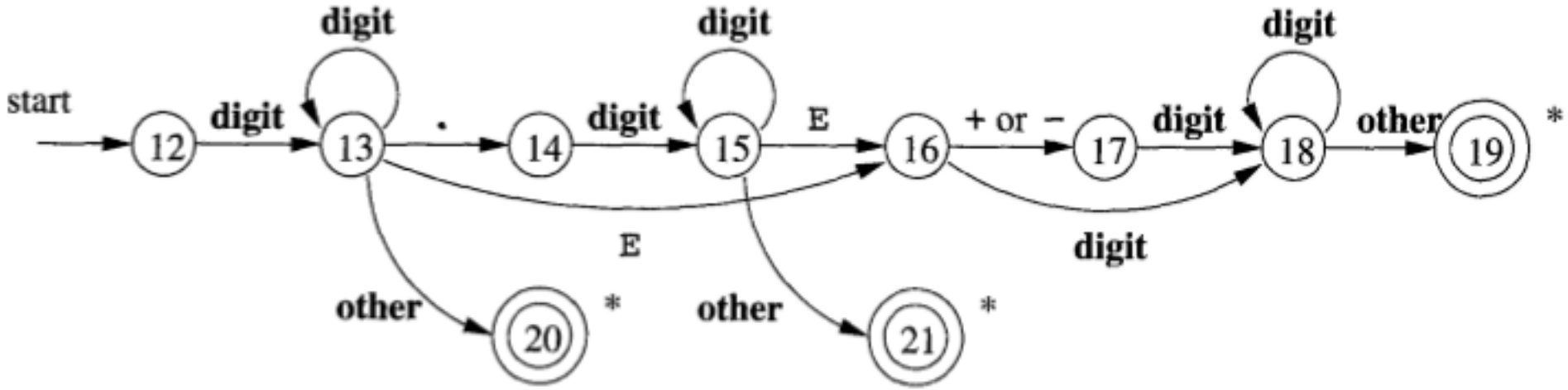


Figure 3.16: A transition diagram for unsigned numbers

Finite automata

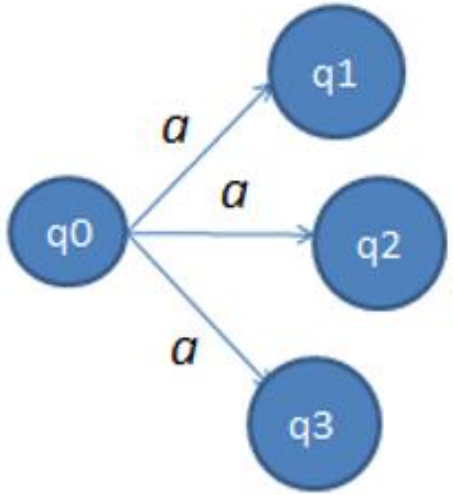
- Finite automata are recognizers; they simply say "yes" or "no" about each possible input string.
- Finite automata come in two types:

1. Nondeterministic finite automata (NFA) .
2. Deterministic finite automata (DFA).

Nondeterministic Finite Automata (NFA)

The Nondeterministic Finite Automata (NFA):

- Have no restrictions on the labels of their edges.
- A symbol can label several edges out of the same state, and ∈, the empty string, is a possible label.



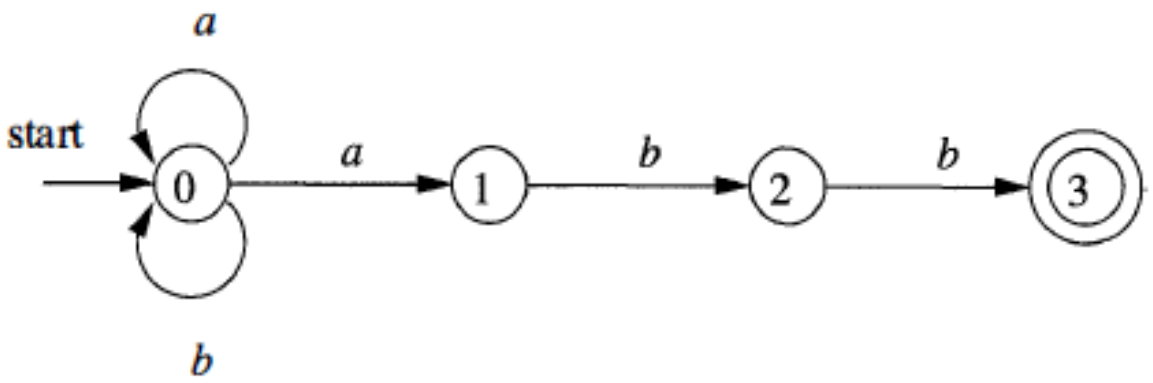
Nondeterministic Finite Automata (NFA)

A nondeterministic finite automaton (NFA) consists of:

1. A finite set of states S.
2. A set of input symbols Σ, the input alphabet. We assume that ∈, which stands for the empty string, is never a member of Σ .
3. A transition function that gives, for each state, and for each symbol in Σ U {∈} a set of next states.
4. A state S_0 from S that is distinguished as the start state (or initial state) .
5. A set of states F, a subset of S, that is distinguished as the accepting states (or final states) .

Nondeterministic Finite Automata (NFA)

- The transition graph for an NFA recognizing the language of regular expression (alb) * abb is shown

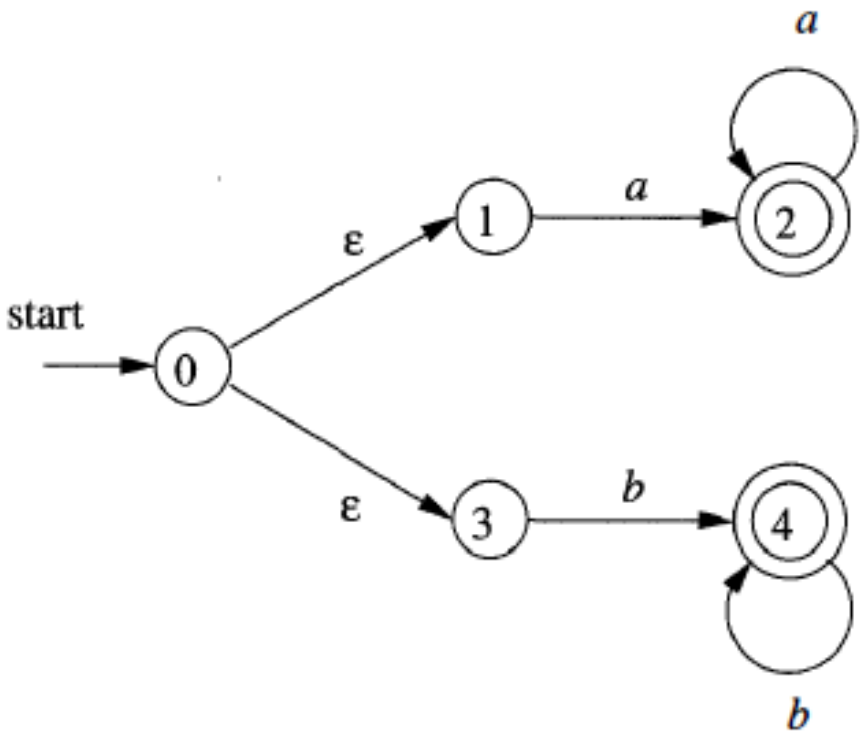


Acceptance of Input Strings by Automata

- An NFA accepts input string x if and only if there is some path in the transition graph from the start state to one of the accepting states, such that the symbols along the path spell out x.
- Note that $\in$ labels along the path are ignored.

Nondeterministic Finite Automata (NFA)

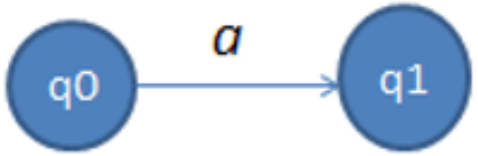
- NFA accepting aa*| bb*



Deterministic Finite Automata (DFA)

The Deterministic Finite Automata (DFA):

1. There are no moves on input ∈, and
2. For each state S and input symbol a, there is exactly one edge out of s labeled a.



From regular expression to NFA then to DFA

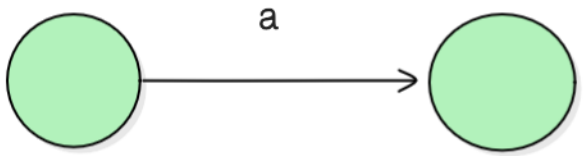
- It is proven (Kleene's Theorem) that Regular expression (RE) and Finite Automata (FA) are equivalent language definition methods.
- Based on this theoretical result practical algorithms have been developed enabling us actually to construct FA's from RE's and simulate the FA with a computer program using Transition Tables.
- In following this progression an NFA is constructed first from a regular expression, then the NFA is reconstructed to a DFA, and finally a Transition Table is built.

From regular expression to NFA then to DFA

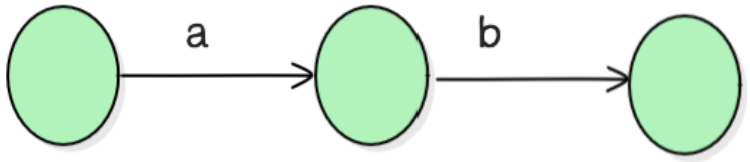
- The Thompson's Construction
- Algorithm is one of the algorithms that can be used to build a Nondeterministic Finite Automaton
- (NFA) from RE, and Subset construction Algorithm can be applied to convert the NFA into a
- Deterministic Finite Automaton (DFA). The last step is to generate a transition table.

Thompson's construction

- This method constructs a regular expression from its components using ε-transitions.
- The ε transitions act as "glue or mortar" for the subcomponent NFA's. An ε-transition adds nothing since concatenation with the empty string leaves a regular expression unchanged (concatenation with ε is the identity operation).
- a

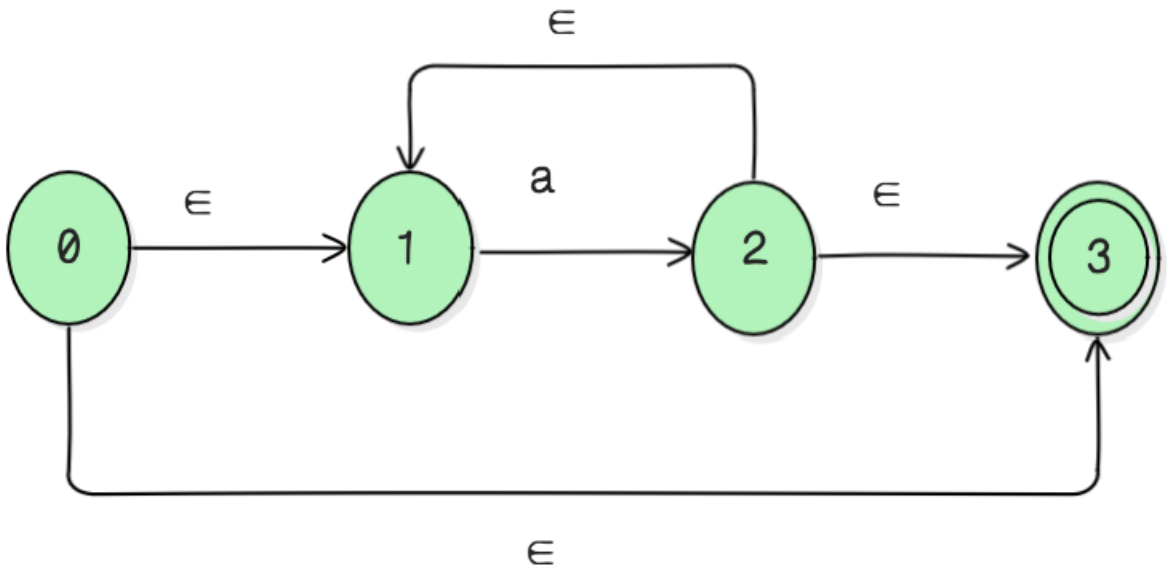


- Concatenation ab



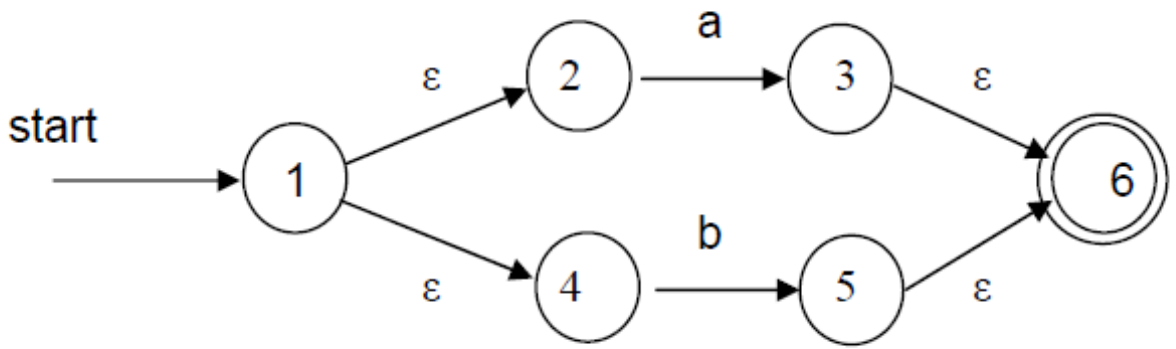
Thompson's construction

- a*



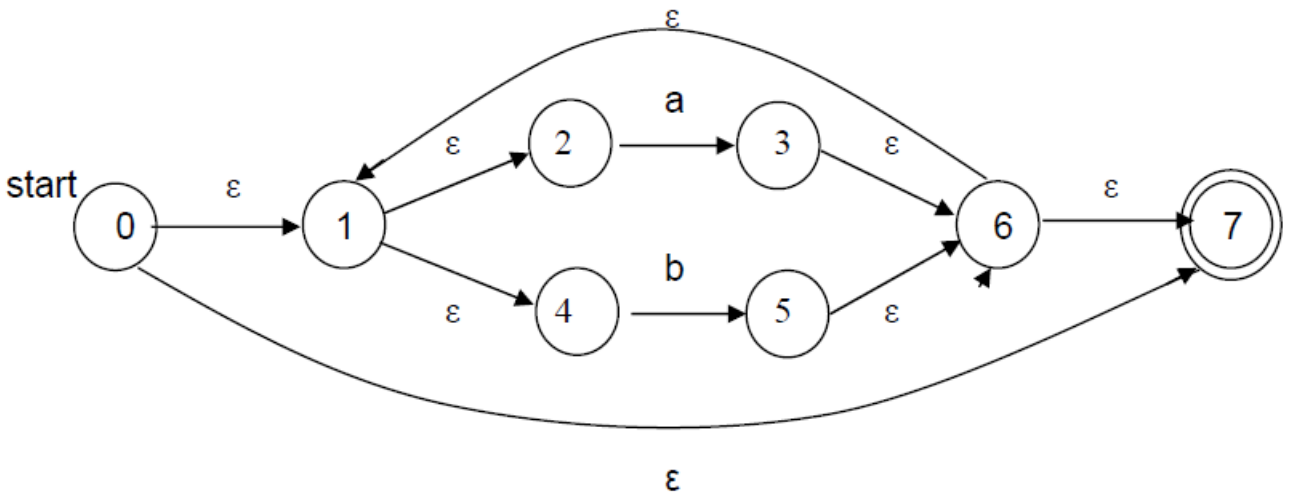
Thompson's construction

- Union : a|b



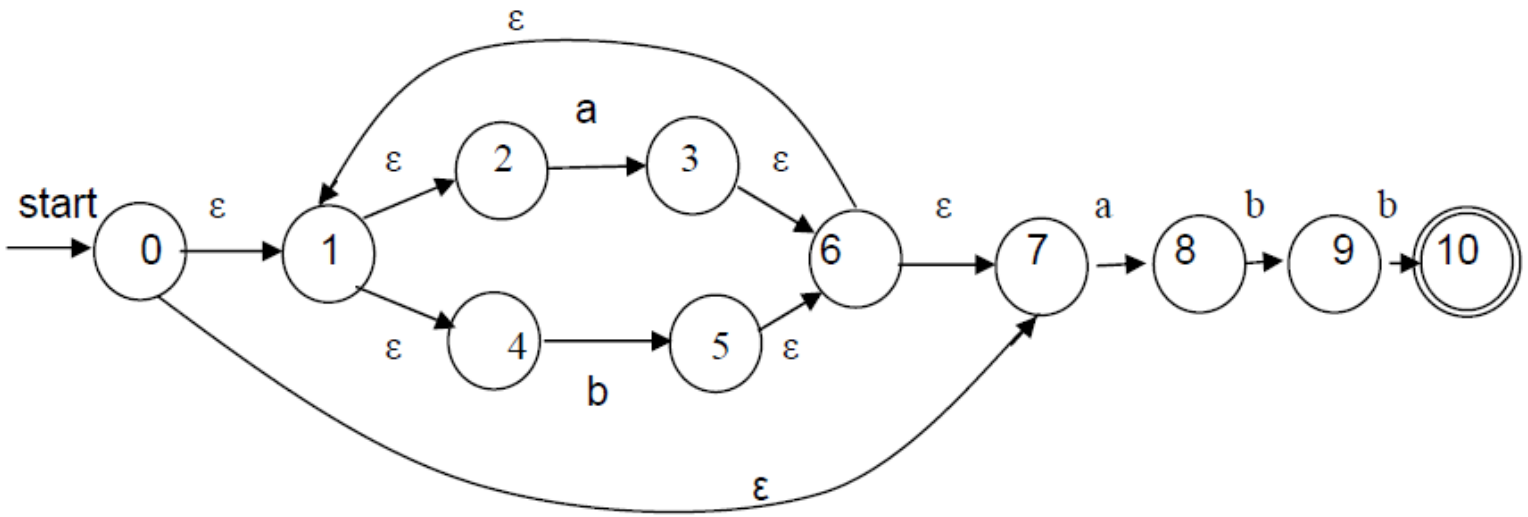
Thompson's construction

- (a|b)*



Example

- Convert the following regular expression into an NFA using epsilon transition (Thompson construction)
- (a|b)*abb



Simulating a DFA

- INPUT: An input string x terminated by an end-of-file character **eof** , a DFA **D** with start state **S0**, accepting states **F**, and transition function **move**.
- OUTPUT: Answer **"yes" if D accepts x; "no" otherwise.**
- METHOD: Apply the algorithm in the next slide to the input string x.
  - ✓The function **move(s, c)** gives the state to which there is an edge from state **s** on input **c**.
  - ✓The function **next Char** returns the next character of the input string x.

Conversion from NFA to DFA

- An algorithm for simulating a DFA

```
s = s0;
c = nextChar();
while ( c != eof ) {
        s = move(s, c);
        c = nextChar();
}
if ( s is in F ) return "yes";
else return "no";
```

Conversion from NFA to DFA

- A Deterministic Finite Automaton (DFA) has at most one edge from each state for a given symbol and is a suitable basis for a transition table. We need to eliminate the ε-transitions by subset construction.

Operation	Description
ε-closure(s)	Set of NFA states reachable from NFA state **s** on ε-transitions alone
ε-closure(T)	Set of NFA states reachable from set of states **T** on ε-transitions alone
move(T,a)	Set of states to which there is a transition on input symbol **a** from some NFA state in **T**

Conversion from NFA to DFA

(Back to the previous example) (a|b)*abb

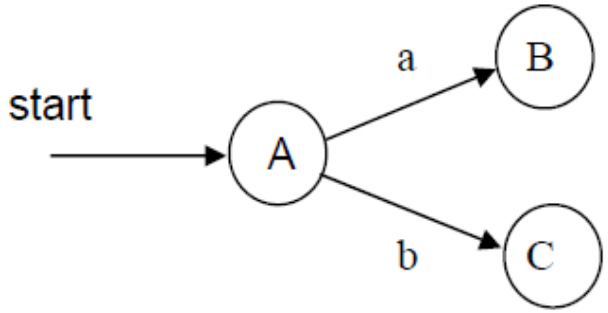
Answer:

1. Begin with the start state 0 and calculate ε-closure(0).
   a. the set of states reachable by ε-transitions which includes 0 itself is { 0,1,2,4,7}. This defines a new state A in the DFA

      A = {0,1,2,4,7}.

   2. We must now find the states that A connects to. There are two symbols in the language (a, b) so in the DFA we expect only two edges: from A on a and from A on b.

      Call these states B and C:



We find B and C in the following way:

Find the state B that has an edge on a from A

a. start with A{0,1,2,4,7}. Find which states in A have states reachable by **a** transitions. This set is called move(A,a) The set is {3,8}:

move(A,a) = {3,8}

b. now do an ε-closure on **move(A,a)**. Find all the states in **move(A,a)** which are reachable with ε-transitions. We have 3 and 8 to consider. Starting with 3 we can get to 3 and 6 and from 6 to 1 and 7, and from 1 to 2 and 4. Starting with 8 we can get to 8 only. So the complete set is {1,2,3,4,6,7,8}. So

ε-closure(move(A,a)) = **B = {1,2,3,4,6,7,8}**

This defines the new state B that has an edge on **a** from A

Find the state C that has an edge on b from A

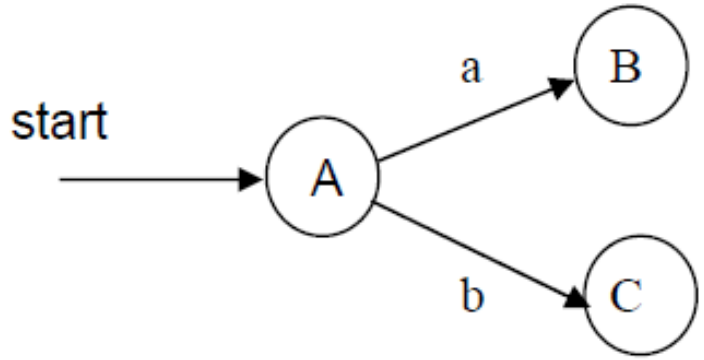
c. start with A{0,1,2,4,7}. Find which states in A have states reachable by **b** transitions. This set is called move(A,b) The set is {5}:

move(A,b) = {5}

d. now do an ε-closure on **move(A,b)**. Find all the states in **move(A,b)** which are reachable with ε-transitions. We have only state 5 to consider. From 5 we can get to 5, 6, 7, 1, 2, 4. So the complete set is {1,2,4,5,6,7}. So

ε-closure(move(A,a)) = **C = {1,2,4,5,6,7}**

This defines the new state C that has an edge on **b** from A



A={0,1,2,4,7}
B={1,2,3,4,6,7,8}
C={1,2,4,5,6,7}

Now that we have B and C we can move on to find the states that have **a** and **b** transitions from B and C.

Find the state that has an edge on a from B

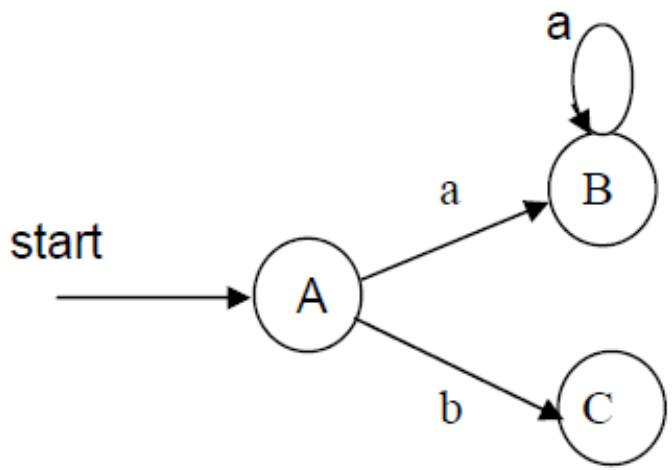
e. start with B{1,2,3,4,6,7,8}. Find which states in B have states reachable by **a** transitions. This set is called move(B,a) The set is {3,8}:

move(B,a) = {3,8}

f. now do an ε-closure on **move(B,a)**. Find all the states in **move(B,a)** which are reachable with ε-transitions. We have 3 and 8 to consider. Starting with 3 we can get to 3 and 6 and from 6 to 1 and 7, and from 1 to 2 and 4. Starting with 8 we can get to 8 only. So the complete set is {1,2,3,4,6,7,8}. So

ε-closure(move(A,a)) = **{1,2,3,4,6,7,8}**

which is the same as the state B itself. In other words, we have a repeating edge to B:



A={0,1,2,4,7}
B={1,2,3,4,6,7,8}
C={1,2,4,5,6,7}

Find the state D that has an edge on b from B

g. start with B{1,2,3,4,6,7,8}. Find which states in B have states reachable by **b** transitions. This set is called move(B,b) The set is {5,9}:

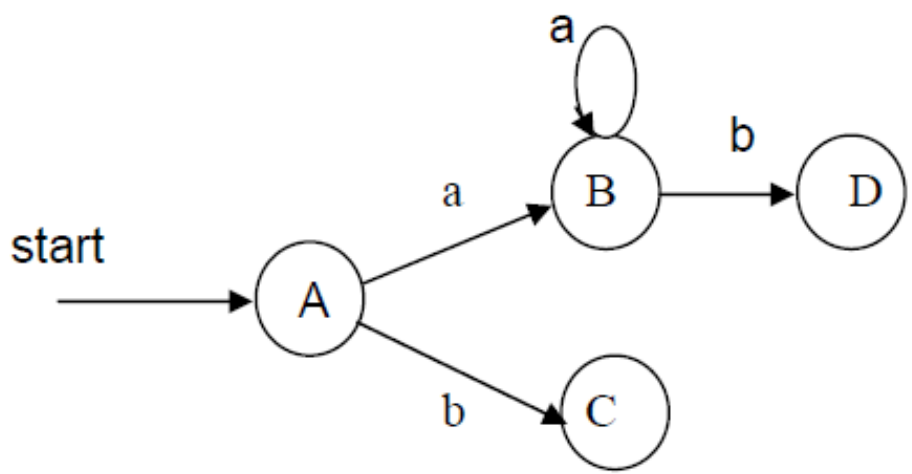
move(B,b) = {5,9}

h. now do an ε-closure on **move(B,b)**. Find all the states in **move(B,b)** which are reachable with ε-transitions. From 5 we can get to 5, 6, 7, 1, 2, 4. From 9 we get to 9 itself. So the complete set is {1,2,4,5,6,7,9}}. So

ε-closure(move(B,a)) = **D = {1,2,4,5,6,7,9}**

This defines the new state D that has an edge on **b** from B

A={0,1,2,4,7},B={1,2,3,4,6,7,8}, C={1,2,4,5,6,7}, D{1,2,4,5,6,7,9}



Find the state that has an edge on a from D

i. start with D{1,2,4,5,6,7,9}. Find which states in D have states reachable by **a** transitions. This set is called move(D,a) The set is {3,8}:

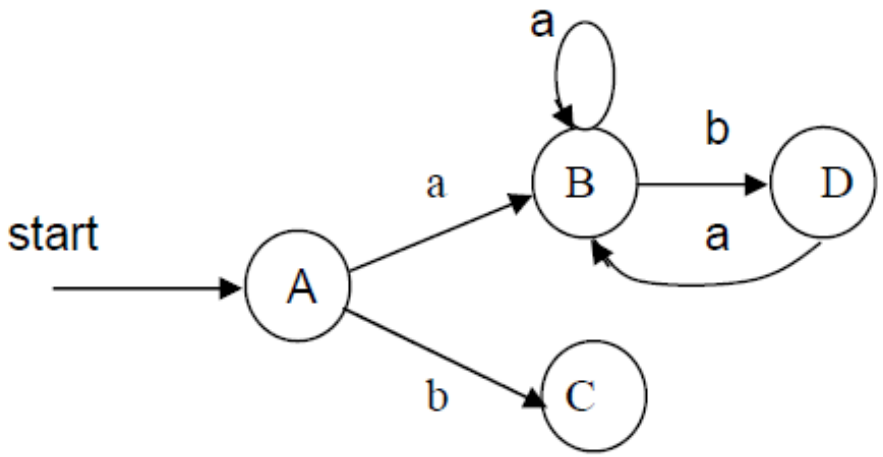
move(D,a) = {3,8}

j. now do an ε-closure on **move(D,a)**. Find all the states in **move(B,a)** which are reachable with ε-transitions. We have 3 and 8 to consider. Starting with 3 we can get to 3 and 6 and from 6 to 1 and 7, and from 1 to 2 and 4. Starting with 8 we can get to 8 only. So the complete set is {1,2,3,4,6,7,8}. So

ε-closure(move(D,a)) = **{1,2,3,4,6,7,8} =B**

This is a return edge to B:

A={0,1,2,4,7},B={1,2,3,4,6,7,8}, C={1,2,4,5,6,7}, D{1,2,4,5,6,7,9}



Find the state E that has an edge on b from D

k. start with D{1,2,4,5,6,7,9}. Find which states in D have states reachable by **b** transitions. This set is called move(B,b) The set is {5,10}:

move(D,b) = {5,10}

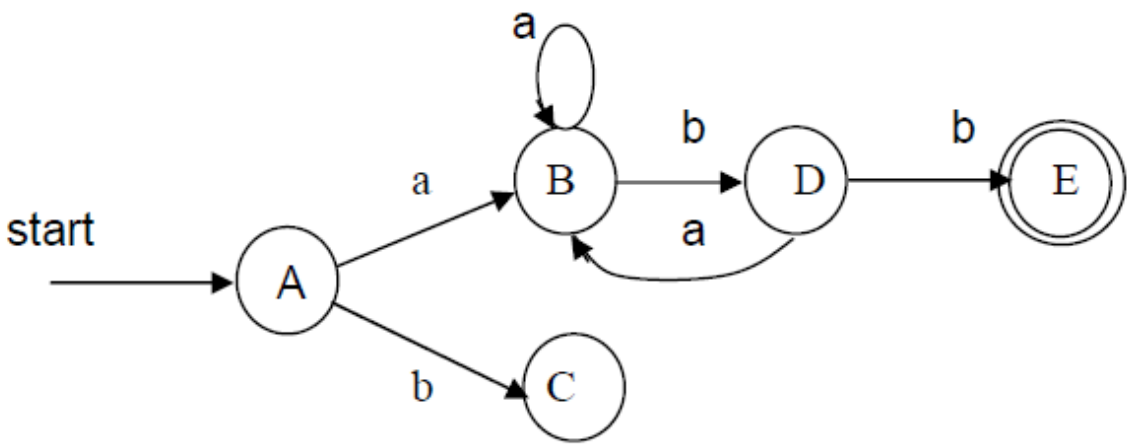
l. now do an ε-closure on **move(D,b)**. Find all the states in **move(D,b)** which are reachable with ε-transitions. From 5 we can get to 5, 6, 7, 1, 2, 4. From 10 we get to 10 itself. So the complete set is {1,2,4,5,6,7,10}}. So

ε-closure(move(D,b) = **E = {1,2,4,5,6,7,10}**

This defines the new state E that has an edge on **b** from D.

Since it contains an accepting state, it is also an accepting state.

A={0,1,2,4,7},B={1,2,3,4,6,7,8}, C={1,2,4,5,6,7}, D={1,2,4,5,6,7,9}, E={1,2,4,5,6,7,10}



We should now examine state C

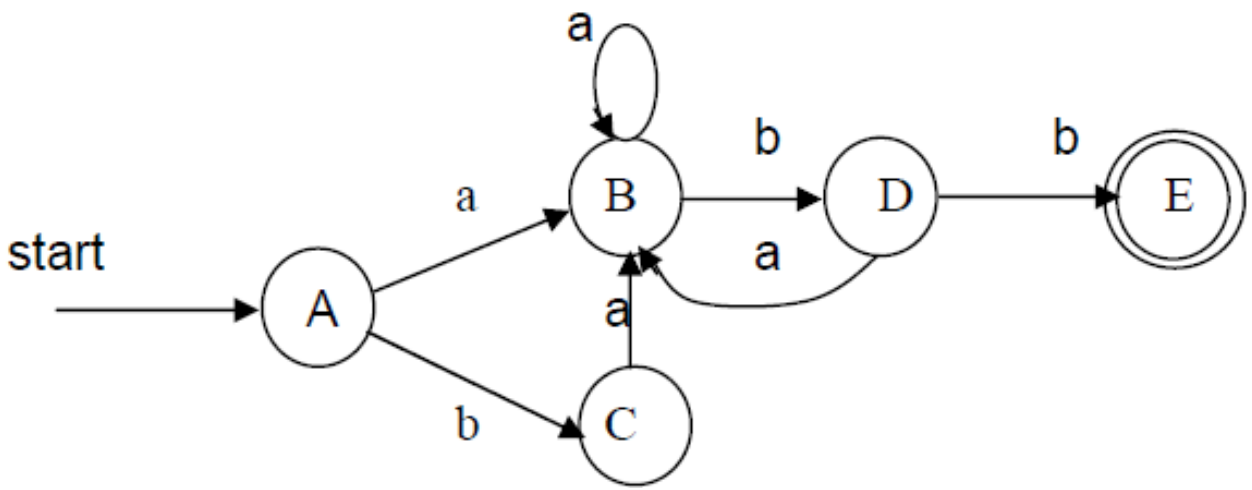
Find the state that has an edge on a from C

m. start with C{1,2,4,5,6,7}. Find which states in C have states reachable by **a** transitions. This set is called move(C,a) The set is {3,8}:

move(C,a) = {3,8}

we have seen this before. It's the state B

A={0,1,2,4,7},B={1,2,3,4,6,7,8}, C={1,2,4,5,6,7}, D={1,2,4,5,6,7,9}, E={1,2,4,5,6,7,10}



Find the state that has an edge on b from C

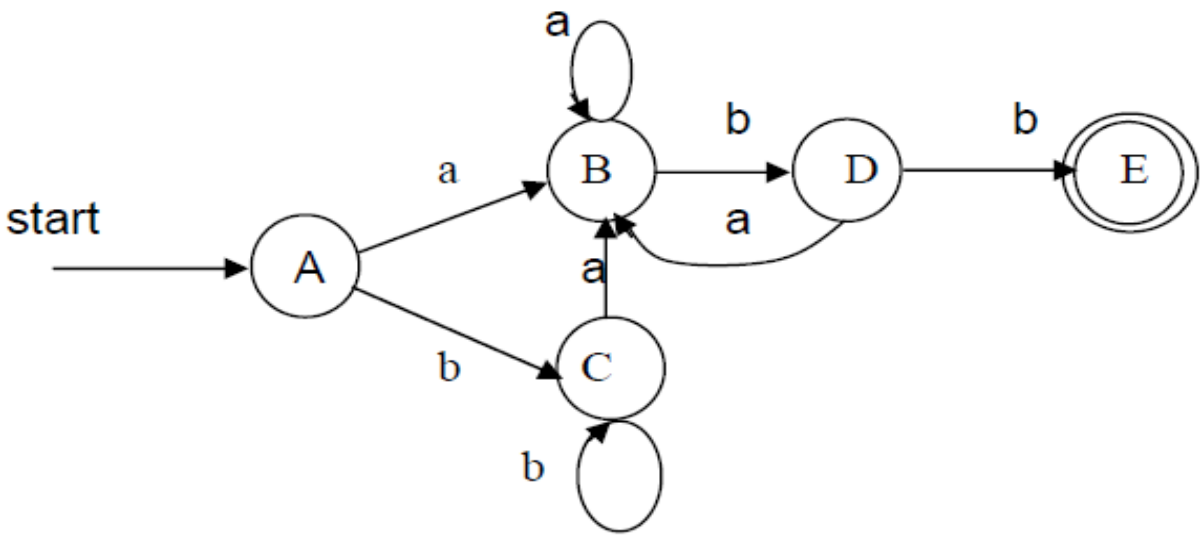
n. start with C{1,2,4,5,6,7}. Find which states in C have states reachable by **b** transitions. This set is called move(C,b) The set is {5}:

move(C,b) = {5}

o. now do an ε-closure on **move(C,b)**. Find all the states in **move(C,b)** which are reachable with ε-transitions. From 5 we can get to 5,6,7,1,2,4. which is C itself So

ε-closure(move(C,b)) = **C**

This defines a loop on C



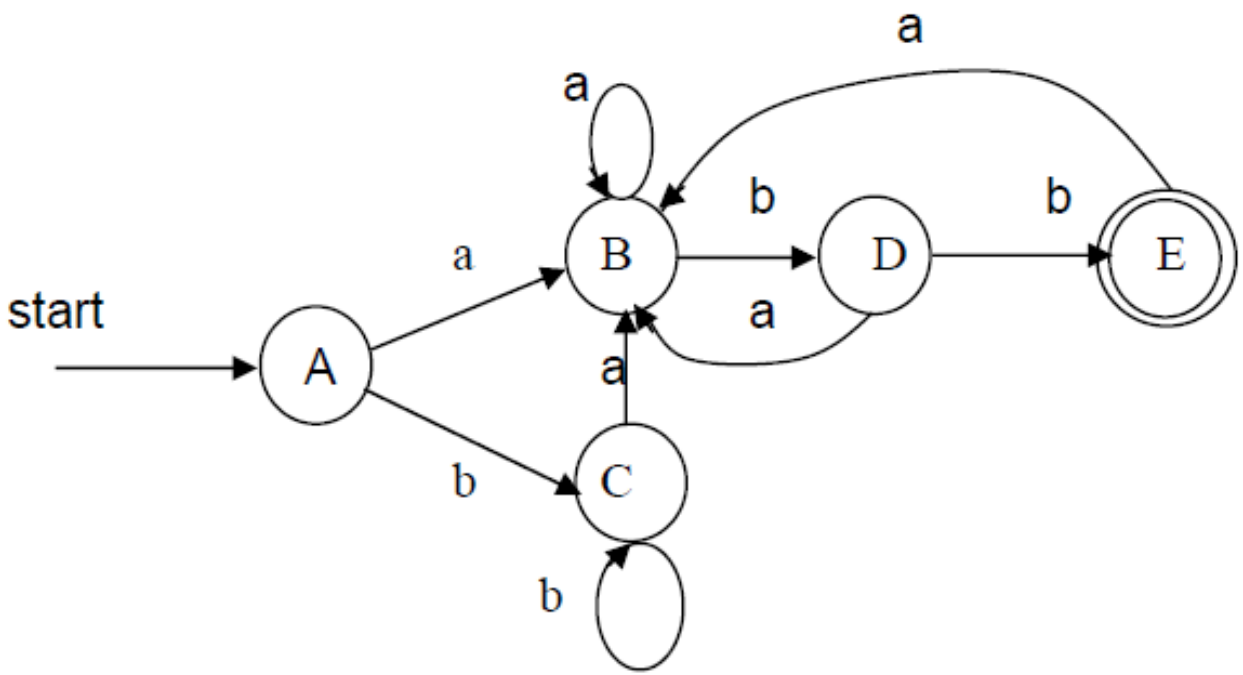
Finally we need to look at E. Although this is an accepting state, the regular expression allows us to repeat adding in more a's and b's as long as we return to the accepting E state finally. So

Find the state that has an edge on a from E

p. start with E{1,2,4,5,6,7,10}. Find which states in E have states reachable by **a** transitions. This set is called move(E,a) The set is {3,8}:

move(E,a) = {3,8}

We saw this before, it's B. So

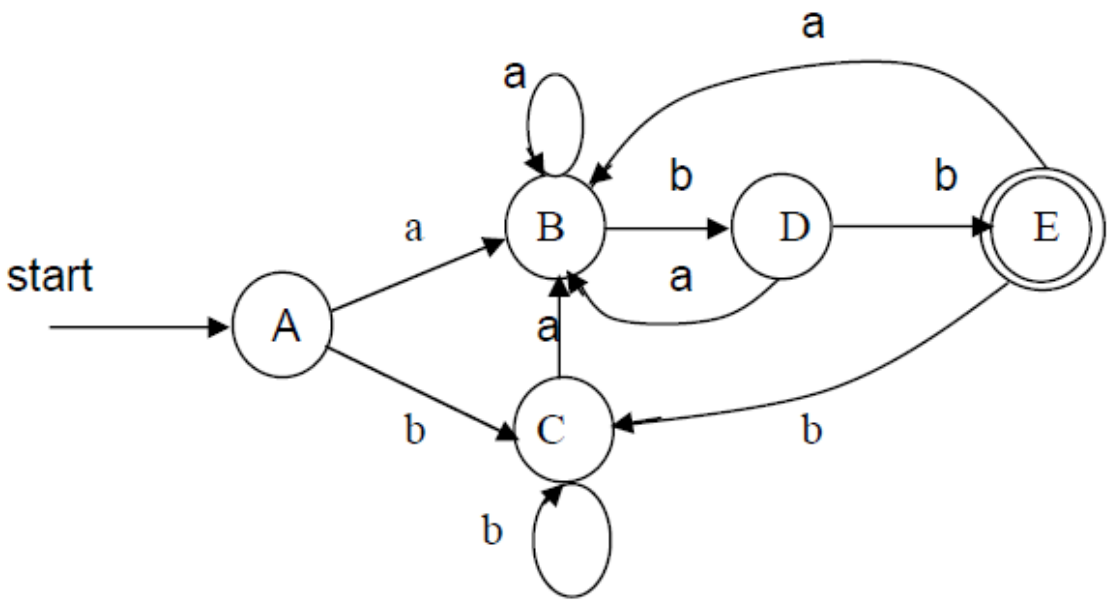


Find the state that has an edge on b from E

q. start with E{1,2,4,5,6,7,10}. Find which states in E have states reachable by **b** transitions. This set is called move(E,b) The set is {5}:

move(A,b) = {5}

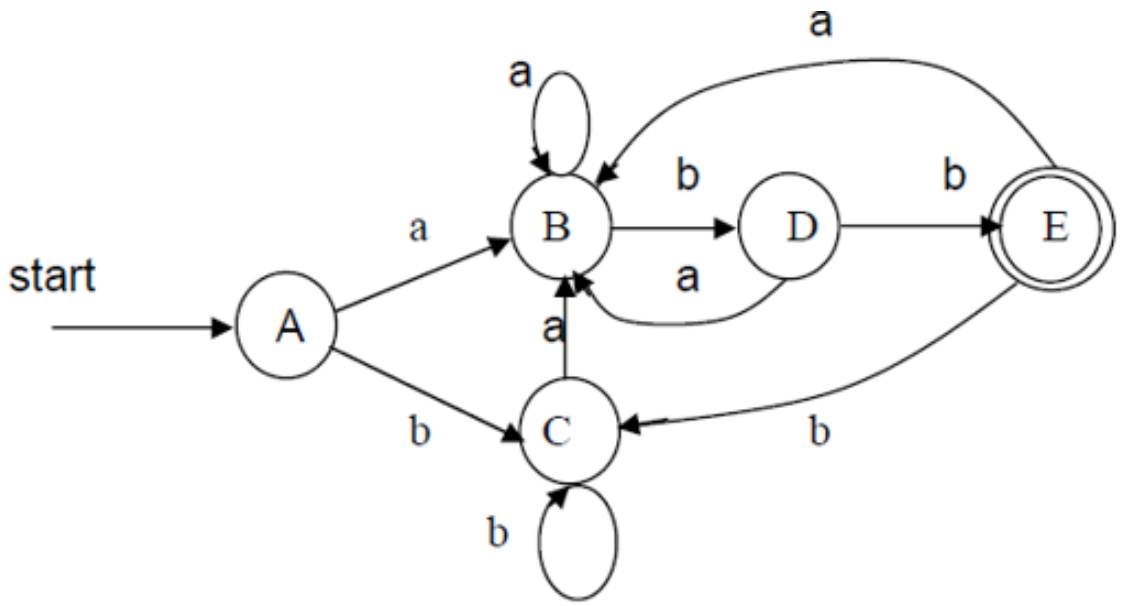
We've seen this before. It's C. Finally



That's it ! There is only one edge from each state for a given input character. It's a DFA. Disregard the fact that each of these states is actually a group of NFA states. We can regard them as single states in the DFA. In fact it also requires **other** as an edge beyond E leading to the ultimate accepting state. Also the DFA is not yet optimized (there can be less states).

Transition table

State	Input **a**	Input **b**
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C



DFA

Thank you

